UNIVERSITY OF PETROLEUM AND ENERGY STUDIES, DEHRADUN

SCHOOL OF COMPUTER SCIENCE

PROJECT SYNOPSIS REPORT

VFRED-Gate: Spec-Aware Numerical Compatibility Qualification of Large Language Model Inference on RISC-V Vector Hardware

Submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

SUBMITTED BY

Name	SAP ID	Roll Number
Anshuman Agrawal	500126680	R2142232078
Ibteshamul Haque	500123421	R2142231557
Anshul Goel	500123668	R2142231828
Aayushi Sinha	500122718	R2142231738

UNDER THE MENTORSHIP OF

Dr. Shaurya Gupta

Associate Professor, School of Computer Science

September 2026 Academic Session 2023–27

Abstract

The RISC-V Vector Extension, version 1.0, defines two distinct instructions for reducing a vector of floating-point values to a scalar. The ordered form, `vfredosum.vs`, is required to accumulate strictly in element order. The unordered form, `vfredusum.vs`, is explicitly permitted to evaluate the sum using any binary reduction tree, with each node of that tree free to round its intermediate result to any implementation-chosen format whose range and precision are at least those of the element format. Because floating-point addition is not associative, this freedom is sufficient for two fully conformant implementations to return different numerical results for the same instruction applied to identical inputs.

Large language model inference is dense with reductions. Root-mean-square normalisation, softmax, every dot product and the whole of scaled dot-product attention are reductions over floating-point values. A difference on the order of 10^{-7} in a logit vector is ordinarily harmless, but under greedy decoding the emitted token is the *argmax* of that vector. Where the top two logits are separated by less than the accumulated rounding difference, the argmax flips, a different token is emitted, and the autoregressive loop thereafter feeds the two executions different inputs. The result is a model that behaves differently after a change to the compiler, the runtime, or the target board, with no defect present anywhere in the system.

Existing work does not close this gap. Determinism-oriented systems eliminate variation at a cost paid uniformly across every reduction in the model. Differential testing tools compare backend outputs but do not classify an observed difference as permitted or prohibited by an instruction set specification, and do not localise it to an instruction. Architectural certification qualifies a processor against its specification and is silent on application behaviour. Benchmarks report that a quality score has moved without attributing the cause.

This synopsis proposes **VFRED-Gate**, a self-hosted release gate that occupies the layer between architectural certification and model evaluation. Given a trusted baseline software stack, a candidate stack, a hash-pinned model, a canary corpus and a declarative behaviour contract, the proposed tool returns a four-valued verdict with a continuous-integration exit code, localises the first causal incompatibility from an emitted token down to a specific machine instruction using teacher-forced replay, samples the space of reduction strategies the specification permits in order to classify an observed difference as conformant or defective, and searches for a minimum-cost selective hardening policy as an alternative to global deterministic execution.

Six research objectives are stated in falsifiable form, together with a preregistration protocol, a three-axis controlled experimental design, a twenty-four week work plan with phase gates, and an explicit register of risks and kill criteria. The report closes with an explicit statement of the claims the project does *not* make.

Keywords: RISC-V vector extension; floating-point reduction order; reproducibility of inference; compiler qualification; behaviour contracts; causal localisation; delta debugging.

Contents

Abstract **i**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background 1
1.2 Reduction order in vectorised hardware 2
1.3 The specification grants the freedom explicitly 2
1.4 Propagation into model behaviour 3
1.5 Problem context: four indistinguishable situations 4
1.6 Scope of the proposed work 4
1.7 Organisation of this report 4

2 Literature Review **6**

2.1 Determinism and reproducibility in inference 6
2.2 Differential testing and backend comparison 6
2.3 Numerical divergence in neural execution 6
2.4 Architectural certification and machine-readable specifications 7
2.5 Compiler treatment of floating-point reductions 7
2.6 Comparative gap analysis 7
2.7 Statement of the research gap 8

3 Problem Statement and Objectives **10**

3.1 Problem statement 10
3.2 Research objectives 10
3.3 Hypotheses 11
3.4 Claims explicitly not made 12

4 Proposed Methodology **13**

4.1 Overview of the proposed system 13
4.2 Behaviour contracts and the compatibility ladder 13
4.3 System architecture 14
4.4 The qualification pipeline 15
4.5 Controlled comparison and the isolation profile 15

4.6 Causal localisation via teacher-forced replay . . . 16
4.7 Sampled legal replay and the portability envelope . . . 17
4.8 Minimum-cost selective hardening . . . 17
4.9 Design decisions taken during the synopsis phase . . . 18

5 System Requirements and Tools 19
5.1 Hardware requirements . . . 19
5.2 Software stack . . . 19
5.3 Pinned configuration for the first release . . . 20
5.4 Scope boundary . . . 21
5.5 Repository layout . . . 21

6 Experimental Design and Evaluation Plan 23
6.1 Three experimental axes . . . 23
6.2 Metrics . . . 24
6.3 Statistical procedures . . . 25
6.4 Preregistration protocol . . . 25
6.5 Threats to validity . . . 26

7 Project Plan 27
7.1 Phase structure and gates . . . 27
7.2 Schedule . . . 28
7.3 Work breakdown structure . . . 29
7.4 Feasibility . . . 29
7.5 Risk register and kill criteria . . . 31

8 Expected Outcomes and Conclusion 32
8.1 Expected deliverables . . . 32
8.2 Definition of done . . . 32
8.3 Broader applicability . . . 33
8.4 Conclusion . . . 33

References 35

A Illustrative Behaviour Contract 36

B Illustrative Verdict Output 37

C Verified Demonstration Programs 39
C.1 Non-associativity of floating-point addition . . . 39
C.2 Sequential versus tree reduction . . . 39
C.3 Absorption and ULP growth . . . 40
C.4 Reduction order changing an argmax . . . 40

D Glossary **42**

List of Figures

Figure 1.1. IEEE 754 binary32 field layout 1
Figure 1.2. Sequential versus tree reduction 2
Figure 1.3. Propagation from rounding difference to behavioural divergence 3

Figure 2.1. Positioning of the proposed contribution 8

Figure 4.1. The compatibility ladder 14
Figure 4.2. Proposed system architecture 15
Figure 4.3. End-to-end qualification pipeline 15
Figure 4.4. Localisation by teacher-forced replay 16
Figure 4.5. Sampled portability envelope 17
Figure 4.6. Two-stage selective hardening search 18

Figure 6.1. The three experimental axes 23

Figure 7.1. Twenty-four week project schedule 28
Figure 7.2. Work breakdown structure 29

List of Tables

Table 1.1. RVV 1.0 floating-point sum reductions 3
Table 1.2. The four outcomes that must be distinguished 4

Table 2.1. Comparative gap analysis of related work 8

Table 3.1. Research objectives 11
Table 3.2. Scope of claims 12

Table 4.1. Verdict model and exit codes 13
Table 4.2. Design decisions and rejected alternatives 18

Table 5.1. Hardware requirements 19
Table 5.2. Software stack and rationale 20
Table 5.3. Scope boundary for the first release 21

Table 6.1. Compiler axis configuration matrix 23
Table 6.2. Evaluation metrics 24
Table 6.3. Statistical procedures 25
Table 6.4. Threats to validity 26

Table 7.1. Phase structure, deliverables and gates 27
Table 7.2. Responsibility allocation 29
Table 7.3. Feasibility: subsystem precedents 30
Table 7.4. Risk register 31

Table 8.1. Expected deliverables 32

List of Abbreviations

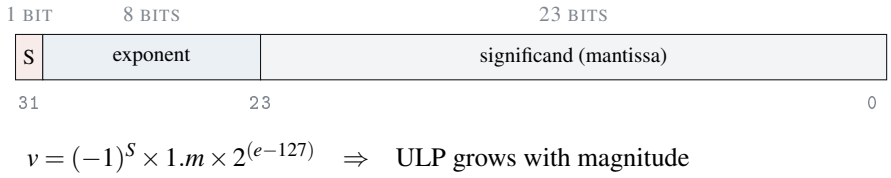
Abbreviation	Expansion
ACT	Architectural Certification Tests (RISC-V)
ADR	Architecture Decision Record
CI	Continuous Integration
DWARF	Debugging With Attributed Record Formats
ddmin	Delta Debugging Minimisation algorithm
ELF	Executable and Linkable Format
FMA	Fused Multiply-Add
GGUF	GPT-Generated Unified Format (quantised model container)
IEEE 754	IEEE Standard for Floating-Point Arithmetic
ISA	Instruction Set Architecture
KV cache	Key–Value cache
LMUL	Vector register group Length MULtiplier (RVV)
LLM	Large Language Model
MoE	Mixture of Experts
RMSNorm	Root Mean Square Normalisation
RVV	RISC-V Vector Extension
SEW	Selected Element Width (RVV)
SHA-256	Secure Hash Algorithm, 256-bit digest
SIMD	Single Instruction, Multiple Data
ULP	Unit in the Last Place
UDB	RISC-V Unified Database
VLEN	Vector register length in bits (RVV)

CHAPTER 1

Introduction

1.1 Background

A single-precision floating-point number is 32 bits wide. One bit is the sign, eight bits hold the exponent, and the remaining twenty-three hold the significand, which is the fractional part. Figure 1.1 shows the layout. The thing to notice is that the significand has a fixed width. That means a float always carries roughly the same number of significant digits, no matter how big or small the number is. So the gap between one representable value and the next one up, which is usually called the unit in the last place or ULP, is not fixed. It gets wider as the numbers get bigger.



$$v = (-1)^S \times 1.m \times 2^{(e-127)} \quad \Rightarrow \quad \text{ULP grows with magnitude}$$

Figure 1.1: How the 32 bits of an IEEE 754 single are divided up. The significand is a fixed width, so the gap between neighbouring values grows with the exponent. Near 2^{19} the gap is 0.0625. Near 10^{20} it is more than 8.8×10^{12}.

IEEE 754 says that every arithmetic operation should work out the exact answer and then round it to the nearest value the format can hold. Taken one at a time, these roundings are tiny and they are all correct. The trouble is that when you do a lot of them, and especially when you change the order in which you do them, the results stop behaving like ordinary real-number arithmetic. The simplest way to see this needs only three numbers:

$$(10^{20} + (-10^{20})) + 3.14 = 3.14, \qquad 10^{20} + ((-10^{20}) + 3.14) = 0.0. \tag{1.1}$$

Both sides of Equation 1.1 are computed correctly at every step. What happens in the second one is that 3.14 is simply too small to register next to 10^{20}. It is less than half the gap between neighbouring floats at that magnitude, so adding it does nothing, and the value is lost before the big cancellation gets a chance to reveal it. Neither answer is wrong. We ran this, along with three more small programs that show related effects, and the actual output is in Appendix C.

1.2 Reduction order in vectorised hardware

A reduction is any operation that takes an array and collapses it to a single number, using an operator like addition or maximum that is associative when applied to real numbers. The way most people first write a sum is a sequential loop, where each addition waits for the previous one to finish. Parallel hardware does not want that chain of dependencies. Instead it adds pairs, then adds the pairs of pairs, and so on, forming a binary tree. Figure 1.2 shows the two side by side. For n elements, the sequential version has a critical path of $n-1$ additions, while the tree has one of about $\log_2 n$. This is why hardware does it. It is a performance choice and there is nothing wrong with it.

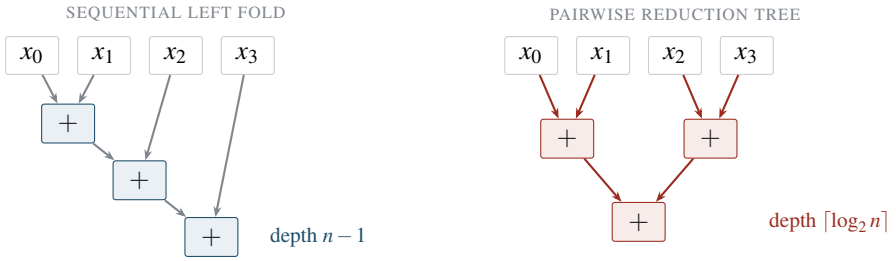


Figure 1.2: The same sum done two ways. The left fold has to wait for each addition to finish. The tree does many at once, which is why vector hardware uses it. On 2^{20} random values in $[0, 1)$ the two give answers about 5.4 apart in `binary32` (Appendix C, Listing C.3).

1.3 The specification grants the freedom explicitly

On most instruction sets, the order in which a vector reduction is carried out is simply not specified. The RISC-V vector extension is different. Version 1.0 defines two separate instructions for summing a vector of floats, and they are summarised in Table 1.1. The ordered one has to add in element order. The unordered one is allowed to use any binary tree, and on top of that, each node in the tree can round its intermediate result to whatever format the implementer chose, as long as the range and precision are at least those of the element type. Different nodes in the same tree are allowed to choose differently [1].

Table 1.1: The two single-width floating-point sum reductions in RVV 1.0, and how much freedom each one has.

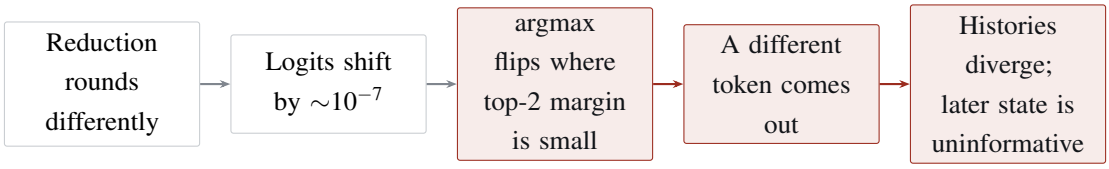
Instruction	Class	Latitude granted by the specification
`vfredosum.vs`	**Ordered**	None. Has to add in element order, starting from the scalar operand. Gives the same result on every implementation, but it brings back the serial dependency chain, so it is slower.
`vfredusum.vs`	**Unordered**	Any binary reduction tree. Each node can round to whatever format the implementer chose, as long as its range and precision are at least those of the element, and different nodes can make different choices.

Why this matters for the project

Take two RISC-V chips. Both pass the architectural certification tests. Both follow the specification in every detail. They can still return different numbers for `vfredusum.vs` on the same inputs. There is no bug in either of them, neither vendor has made a mistake, and because of that, none of the existing quality checks would ever flag it.

1.4 Propagation into model behaviour

A transformer doing inference is doing reductions almost constantly. RMSNorm sums squares. Softmax takes a maximum and then sums exponentials. Every dot product and every matrix multiplication is a sum of products. Attention is built out of both. When the model is decoding greedily, the token it emits is whichever entry in the logit vector is largest. So if the top two logits happen to be very close, even a very small numerical difference can change which one wins. Figure 1.3 shows how it spreads from there.



What this means for diagnosis. After the split, the two runs are reading different token histories. Comparing tensors past this point tells you about the *effect* of the divergence, not what caused it. Section 4.6 deals with this.

Figure 1.3: How a legal difference in reduction order turns into different generated text. Two runs can also produce the same tokens for a while even though their hidden states already differ. That kind of divergence is not gone, just not visible yet, and it can show up many tokens later.

1.5 Problem context: four indistinguishable situations

If all you can see is that two stacks produce different output, you cannot tell which of four quite different things has happened. Table 1.2 lays them out. Each one calls for a different response, and the fact that current tools lump them together is the problem this project is trying to fix.

Table 1.2: Four situations that look the same from outside but need different responses. Most existing tools report all four as "the outputs differ".

Situation	Characterisation	Required verdict
Nonconformant implementation	The candidate violates the ISA or the declared compiler and runtime semantics.	`FAIL` (defect)
Conformant but contract-breaking	The candidate is completely legal, but it breaks the declared behaviour contract.	`FAIL` (compatibility)
Different but acceptable	The values are different, but every declared rule still passes.	`PASS` (with evidence)
Insufficient evidence	Unstable runs, missing traces, too few samples, or differences that were not controlled for.	`INCONCLUSIVE`

1.6 Scope of the proposed work

This document is a synopsis. We have not implemented any part of the system described in it. The architecture diagrams are designs. The terminal transcript in Appendix B is a mock-up of what we want the output to look like, and it is marked as such. What we have actually done at this stage is study the specification, survey the related work, write down objectives that can fail, fix the experimental configuration, record the design decisions we have made along with the options we rejected, and build a schedule with gates and kill criteria.

1.7 Organisation of this report

Chapter 2 covers the related work and where the gap is. Chapter 3 gives the problem statement and the objectives. Chapter 4 describes the method and the system design. Chapter 5 lists the requirements, the tools we plan to use, and what is out of scope. Chapter 6 sets out the experiments and how we will evaluate them. Chapter 7 has the

schedule, the work breakdown, who owns what, and the risk register. Chapter 8 describes what we expect to deliver and concludes.

CHAPTER 2

Literature Review

2.1 Determinism and reproducibility in inference

There has been a fair amount of recent work on nondeterminism in LLM serving. Batch-invariant kernels make the output of a request independent of what else happens to be in the same batch, so a prompt gets the same answer whether the server is busy or idle [2]. A related line of work fixes the structure of reductions across tensor-parallel configurations, so that changing the number of GPUs does not change the numerics [3]. LLM-42 goes further and provides fully deterministic inference, using a technique it calls verified speculation, and accepts some overhead to get there [4].

All of these are trying to do something different from what we want to do. Their goal is to remove variation altogether, and they pay for that on every reduction in the model, whether or not that particular reduction ever mattered. None of them lets the person deploying the model say what level of difference they can actually tolerate, and none of them checks whether a given difference was allowed by the instruction set.

2.2 Differential testing and backend comparison

NVIDIA's Polygraphy compares the outputs of a network run on different backends and can work out which TensorRT tactic is responsible when the outputs disagree [5]. In terms of workflow, this is the closest existing tool to what we are proposing. Where it differs is in three places. It does not say whether a difference is permitted or forbidden by any ISA specification. It does not trace a difference to a particular machine instruction with source and symbol information. And it does not look for the cheapest way to fix the problem.

2.3 Numerical divergence in neural execution

The fact that reordering an equivalent reduction can change model behaviour has already been shown empirically for sparse mixture-of-experts models, where the order in which expert outputs are aggregated was found to propagate into visible differences in what the model produces [6]. We are relying on that result. We are not claiming it as our own.

What that work does not offer is a way to use the observation at release time: there is no contract format, no verdict, and no attribution down to the instruction.

2.4 Architectural certification and machine-readable specifications

RISC-V International maintains a set of architectural certification tests that check an implementation against the ratified specification, and a Unified Database, which is a machine-readable description of the architecture that documentation and tests can be generated from [7, 8]. Both of these work at the level of the processor. They do not say anything about what happens to an application running on a processor that has passed. That is not a gap in them. It is just not what they are for. But it is the reason a perfectly legal implementation choice can change what a model does without anything noticing.

2.5 Compiler treatment of floating-point reductions

LLVM's own documentation on auto-vectorisation points out that vectorising a floating-point reduction requires the compiler to reassociate the additions, which it will only do under relaxed floating-point flags or when the target instruction itself allows it [9]. For us the practical consequence is that you cannot predict which reduction instruction ends up in the binary by looking at the source. It depends on the compiler, the version, the optimisation flags, and how the loop was written. So we do not try to predict it. We disassemble the binary and look. That decision is recorded in Table 4.2.

2.6 Comparative gap analysis

Table 2.1 puts each of the systems above next to the requirements of what we want to build.

Table 2.1: What each of the neighbouring systems does, and what it does not cover.

System / work	Established capability	Requirement left unmet
Batch invariance [2]	Output independent of batch composition under supported conditions	Cross-stack qualification against a declared behaviour contract
Tensor-parallel invariance [3]	Invariant reduction structure across parallel degrees	Arbitrary baseline/candidate pairs; per-site minimum-cost repair
LLM-42 [4]	Deterministic inference via verified speculation, selective overhead	Release-time evidence; low-level root-cause localisation
Polygraphy [5]	Backend output comparison; isolation of the responsible tactic	ISA-level legality classification; instruction-level attribution and repair
MoE reduction study [6]	Demonstrates causal behavioural divergence from reduction order	Product workflow, CI verdicts, implementation-to-instruction diagnosis
ACT and UDB [8, 7]	Machine-readable ISA description; processor-level certification	Application-level model behaviour under a legal implementation choice

2.7 Statement of the research gap

There are two separate ideas of correctness in use here and nothing joins them up. One asks whether the processor and the compiler followed the rules in the specification. The other asks whether the model still does what the person who deployed it was relying on. A new release can pass the first test and fail the second, and as far as we can find, nothing in any published workflow catches that combination. Figure 2.1 shows where we think our contribution sits.

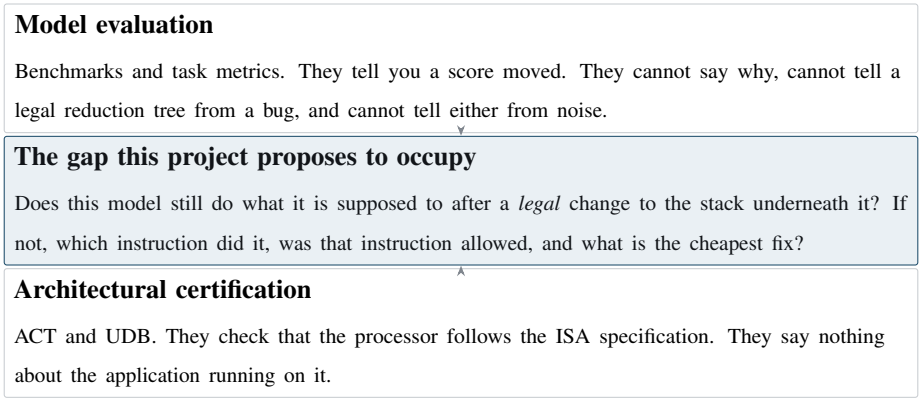


Figure 2.1: Where the proposed system sits: between certification of the processor and evaluation of the model. Neither of the other two layers answers the question in the middle.

Three specific things do not exist yet:

1. a way to write down, in a form a machine can read, what “compatible” means for a given model and release. At the moment this is either an unwritten convention or a threshold buried in somebody’s script;
2. a way to classify an observed numerical difference as legal or illegal against the specification; and
3. a way to get from a symptom at the level of model behaviour to a specific machine instruction, with a check that the instruction really was the cause and not just the first thing that looked different.

CHAPTER 3

Problem Statement and Objectives

3.1 Problem statement

Problem statement

A new software or hardware stack for running a large language model can be completely legal from an architectural point of view and still change the behaviour that the model's deployer was depending on. This is possible because the RISC-V vector extension explicitly lets an unordered floating-point reduction choose its own evaluation tree and intermediate precision. At present there is no release-time mechanism that (i) accepts a written statement of the behaviour that has to be preserved, (ii) decides whether a difference that has been observed is permitted by the instruction set, (iii) traces that difference to a specific machine instruction and confirms the instruction was the cause, and (iv) finds the least expensive change that brings the behaviour back.

3.2 Research objectives

We have written the objectives as questions rather than as things we intend to do, because a question can have the answer "no". Table 3.1 lists them along with how each one will be evaluated.

Table 3.1: The six objectives, each written so it can fail, and how each will be judged.

ID	Theme	Question	Evaluated by
O1	Qualification efficiency	Can a sensitivity-selected canary suite predict full-suite contract failures at a small fraction of the execution cost?	Recall against a frozen holdout suite
O2	Spec-aware portability	Can sampled legal reduction-tree strategies predict the behaviour of an unseen conformant implementation?	Envelope containment rate on a second board
O3	Causal localisation	Can adaptive tracing identify the first causal layer, operator and instruction using far less capture than dumping every tensor?	Capture volume ratio; toggle-confirmation rate
O4	Minimum-cost compatibility	Can selective ordered or widened reductions restore the contract more cheaply than global deterministic execution?	Overhead of selective versus global policy
O5	Axis separation	Can the experimental controls distinguish a compiler code-generation difference from implementation-specific unordered semantics?	Agreement across forced-policy control configurations
O6	Transfer	Which contract, canary and localisation components survive a change of runtime or a distributed backend?	Component reuse audit against a second runtime

Objective O4 is the one that decides it

If it turns out that selective hardening is not cheaper than simply making everything deterministic, then the practical case for this tool disappears, because it would be a slower version of something that already exists. So O4 is where the project can fail in a way that is still worth reporting. If the answer is no, we will say so rather than quietly drop it.

3.3 Hypotheses

H1 If we pick canary prompts by how narrow the gap is between their top two logits, a small suite will catch most of the contract failures that the full suite would catch, at a small fraction of the cost.

H2 For a given reduction site with captured operands, the value produced by conformant hardware will fall inside the envelope we get by sampling legal reduction trees and intermediate precisions.

H3 The set of reduction sites that actually have to be hardened to restore a contract will be much smaller than the set of all reduction sites the model touches during inference.

H4 If both toolchains are forced to use the same reduction policy, any behavioural difference that came from reduction-order choice will go away, and whatever is left will be due to other code-generation decisions.

3.4 Claims explicitly not made

We think a proposal like this is only useful if it says up front what it is not claiming, rather than working that out later under pressure. Table 3.2 lists both sides.

Table 3.2: What we are not claiming, and what we are.

Not claimed	Claimed as contribution
The first demonstration that reduction order can alter LLM behaviour. That has already been shown for mixture-of-experts inference [6].	A sampled portability envelope, built against the specification and tied to real model behaviour rather than synthetic kernels.
The first deterministic inference system. Several already exist [2, 3, 4].	Canary selection guided by sensitivity, which cuts qualification cost without losing failures.
The first backend output comparator [5].	Localisation from a behavioural symptom down to a machine instruction, confirmed by intervention rather than correlation.
A complete proof of every result the unordered reduction could give. The envelope is *sampled*, as Section 4.7 explains.	Minimum-cost selective hardening as an alternative to global deterministic execution.
A replacement for RISC-V architectural certification.	A machine-readable release contract and evidence format linking a stack change to a model-level guarantee.
A legal or regulatory compliance certification product.	

CHAPTER 4

Proposed Methodology

4.1 Overview of the proposed system

We are proposing VFRED-Gate as a self-hosted command-line tool that acts as a release gate. You give it a baseline stack that you trust, a candidate stack that you are thinking about shipping, a model pinned by hash, a corpus of canary prompts, and a behaviour contract. It gives you back a verdict, an exit code that a CI system can act on, and a bundle of evidence that cannot be altered after the fact. We have made it self-hosted on purpose. The models, binaries and hardware traces in a real qualification run are usually a company's unreleased product, and a tool that required uploading them would not get used. Table 4.1 defines the four possible verdicts.

Table 4.1: The four possible verdicts. `ERROR` is kept separate from `INCONCLUSIVE` because the tool breaking is a different kind of event from the evidence being unclear.

Verdict	Exit code	Condition
`PASS`	`0`	Every rule declared in the behaviour contract is satisfied.
`FAIL`	`2`	One or more declared rules are violated.
`INCONCLUSIVE`	`3`	Not enough evidence. Blocks the release by default.
`ERROR`	`4`	The tool itself failed.

Fail closed

Missing or weak evidence must never turn into a `PASS` by default. If the baseline and candidate differ in any way that we did not set up on purpose as part of the experiment, the verdict is `INCONCLUSIVE`. The reason we are strict about this is that letting one small uncontrolled difference through is exactly how a study stops measuring the hardware and starts measuring its own setup.

4.2 Behaviour contracts and the compatibility ladder

The central design decision in this proposal is that "compatible" should not be a yes-or-no question. We treat it as a ladder with six rungs, shown in Figure 4.1. Somebody

deploying a model might genuinely need rung 3 and have no interest at all in rung 1. If the model routes to the same expert and emits the same token, a change in the thirteenth significant digit is not something that should block a release. A behaviour contract is a versioned file, validated against a schema, in which the deployer says which rung they need and what thresholds apply. It goes through the same review process as source code. There is an annotated example in Appendix A.

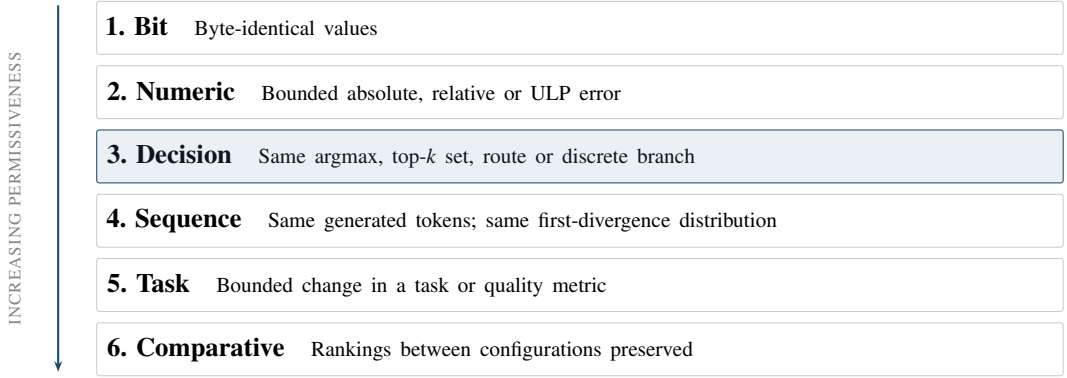


Figure 4.1: The six levels of compatibility a contract can ask for. Determinism systems push every deployment to level 1 and everyone pays for it. A behaviour contract instead says which level is actually needed, so bit-for-bit identity becomes one option among several rather than the definition of correct.

4.3 System architecture

The design has eleven components in three tiers, sitting under a single controller and on top of a shared artifact store. Figure 4.2 shows the arrangement. For the first version we intend to put all of these in one Python package. Even so, we are defining the interfaces between them explicitly, so that a second runtime adapter can be added later without having to touch the contract engine.

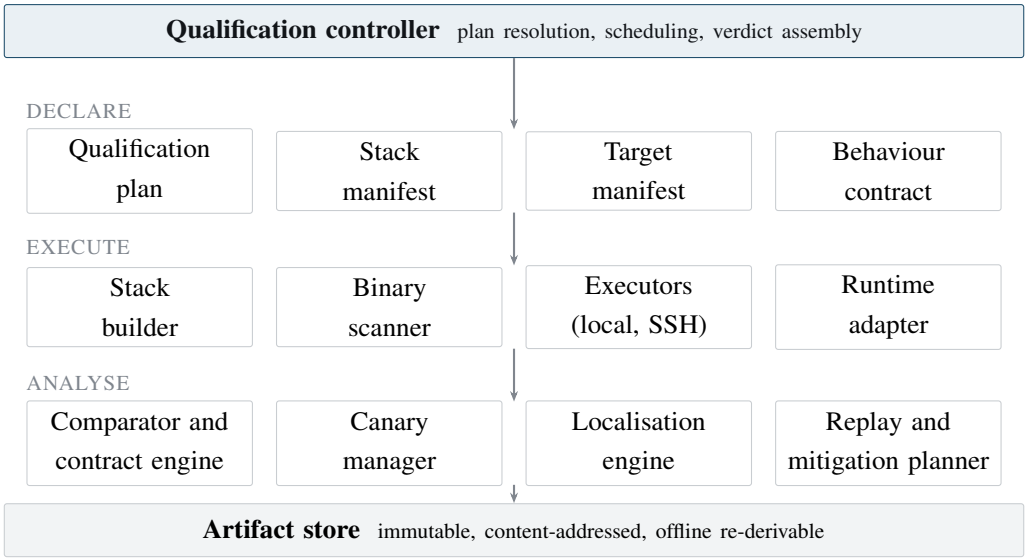


Figure 4.2: The proposed architecture. *This is a design. Nothing here has been implemented.* The controller reads the qualification plan and puts the verdict together. Every intermediate artefact goes into a content-addressed store, so the final report can be rebuilt offline without any hardware.

4.4 The qualification pipeline

Figure 4.3 shows the flow we intend to build, from two source trees at one end to a verdict, and if needed a mitigation, at the other.

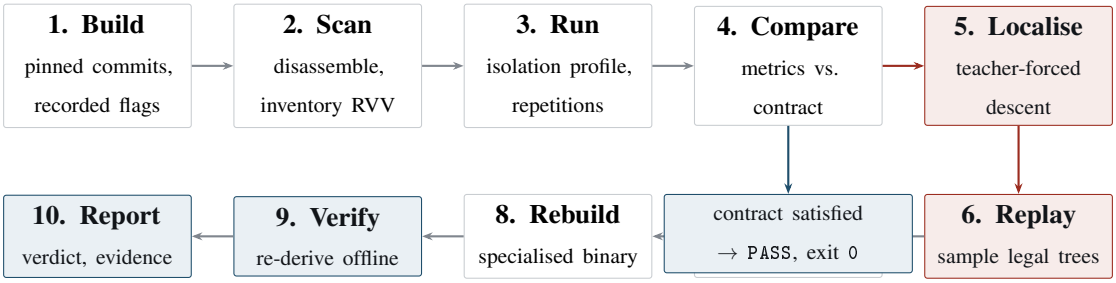


Figure 4.3: The pipeline we intend to build. Stages 5 to 8 only run if the contract is violated. Stage 9 rebuilds every reported figure from the checksummed artefacts with no hardware attached, which is what lets somebody audit the evidence months later.

4.5 Controlled comparison and the isolation profile

A comparison between two stacks only means something if everything we did not mean to change is actually held constant. The protocol we intend to follow is:

1. **Reproducible build.** Both stacks get built from pinned commits, with every compiler

flag written into the stack manifest and hashed. Debug information is added in a way that does not change the optimisation target.

2. **Instruction inventory.** Both binaries get disassembled and the relevant vector instructions counted. We do this before anything else, because without it any claim about which reduction actually ran is just a guess.
3. **Isolation profile.** One thread, pinned to one core, fixed context length, fixed token budget, prompts supplied as pre-tokenised IDs so the tokenizer cannot introduce drift, and nothing else running on the target.
4. **Stability first.** Every case is run several times. Cases that turn out to be unstable are set aside and reported, rather than averaged into the rest.
5. **Screen, then confirm.** A cheap pass over everything, then a more careful pass over only the cases that survived it.

4.6 Causal localisation via teacher-forced replay

This is the part of the method that everything else depends on. As we said in Section 1.4, once the two executions have emitted different tokens they are being fed different inputs, so any comparison of their internal state after that point is looking at the result of divergence, not the cause of it. Our procedure, which Figure 4.4 contrasts with the naive approach, uses the free-running comparison for detection only. Once we know a divergence happened, we take the prefix on which both stacks still agree and replay it in teacher-forced mode, so that each stack is given exactly the same token history. After that, the only thing that differs between the two runs is the arithmetic.

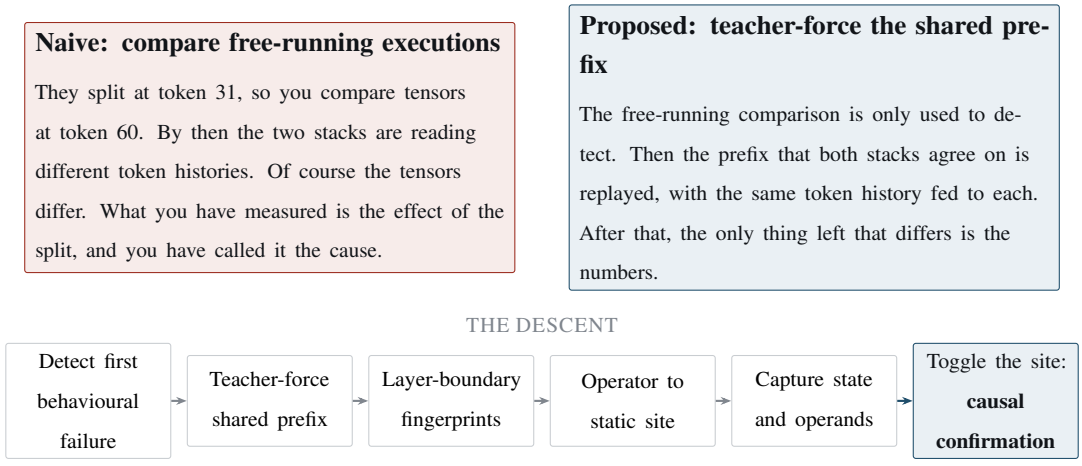


Figure 4.4: On the left, the comparison that is confounded. On the right, the one that is controlled. The last step is what separates this from ordinary differential debugging: we flip the reduction policy at the site we found, and the difference has to go away. "The first tensor that differs" is only a correlation. Flipping one site and watching the symptom disappear is a test.

4.7 Sampled legal replay and the portability envelope

Once we know which site is responsible, we ask a different question: given these particular operands, what results would the specification allow at this site? We capture the operands, generate a sample of legal reduction trees at several permitted intermediate precisions, and collect the results into an envelope. Then we check whether the value the hardware actually produced falls inside it. Figure 4.5 shows the idea.

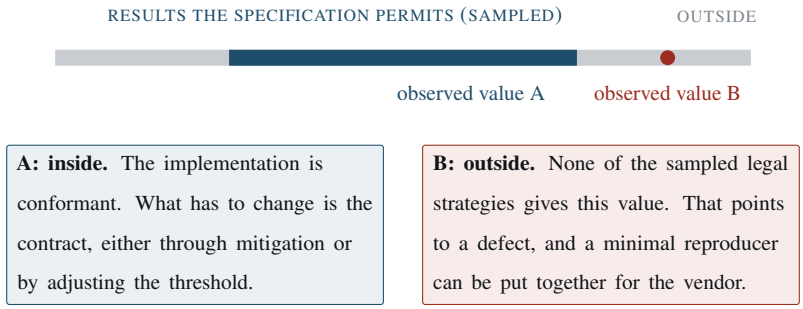


Figure 4.5: Checking whether a value the hardware produced falls inside the set of results the specification allows at that site.

The sampling caveat

We build the envelope by sampling legal reduction strategies, not by listing all of them. The number of legal strategies is far too large for that. So what we will be able to say is that an observed value is consistent with the specification. What we will not be able to say is that no conformant implementation could have produced a different value. We want to be clear about that limit now so that it is not mistaken for a stronger claim later.

4.8 Minimum-cost selective hardening

The obvious fix is to force every reduction into ordered mode. That works, but you pay for it at every reduction in the model, including all the ones that had nothing to do with the problem. What we propose instead is a two-stage search, shown in Figure 4.6. Stage A finds a small set of sites that is enough to make the contract pass, using delta debugging to shrink it down [10]. Stage B then searches within that reduced set for the cheapest option, and rebuilds the winner as a clean binary before doing any performance measurement at all.

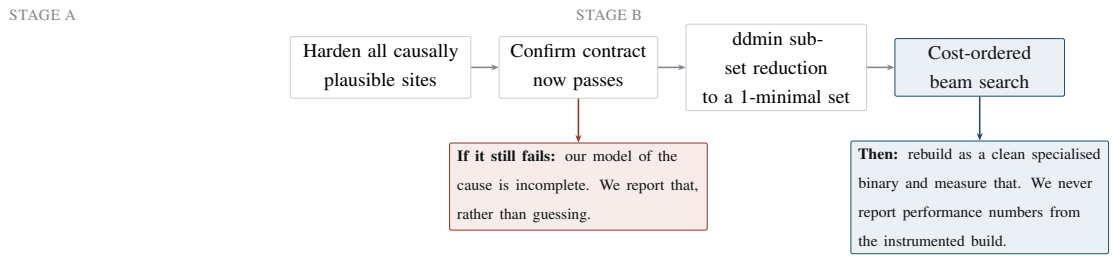


Figure 4.6: Searching for the cheapest hardening policy. Four configurations get compared under one performance protocol: the baseline, the candidate as-is, everything forced ordered, and the selective policy. Objective O4 asks whether the fourth beats the third.

4.9 Design decisions taken during the synopsis phase

Table 4.2 records six decisions where we did not take the obvious option, and why. If any of these is reversed later, it will need an Architecture Decision Record and a new version of this document.

Table 4.2: Decisions we have already made, and the alternative we turned down in each case.

Decision	Selected	Rejected	Rationale
Product form	Self-hosted CLI and artefacts	Hosted dashboard first	The assets are confidential, and a dashboard would eat schedule without testing the hard part.
First execution mode	One thread, batch one, greedy	Production concurrency	Lets us find the ISA-level cause before scheduling effects are layered on top.
Compiler treatment	Scan the emitted binaries	Assume documented GCC/LLVM behaviour	Emission varies with version, flags and source shape [9].
Instrumentation	Tiered, adaptive capture	Dump every tensor	Storage and runtime cost. Narrow down first, then capture in detail.
Localisation	Free-run to detect, teacher-force to diagnose	Compare free-running states	Keeps the model inputs identical while diagnosing (Section 4.6).
Portability model	Sampled legal envelope	Claim a complete semantic proof	Honest about how much of the permitted space we can actually cover.

CHAPTER 5

System Requirements and Tools

5.1 Hardware requirements

Table 5.1 lists the hardware we need. The second RVV implementation is the one item on the list that we do not fully control, and it matters more than anything else on it. Section 7.5 says what we will do if we cannot get one.

Table 5.1: Hardware the project needs. The second RVV implementation is the external dependency that matters most, and Section 7.5 says what we do without it.

Resource	Role	Requirement and rationale
Development host	Build, analysis, reporting	x86-64 Linux, 16 GB RAM, RISC-V cross-toolchains. No GPU needed.
Primary RVV target	Principal execution target	A RISC-V board with RVV 1.0, at least 4 GB RAM, reachable over SSH, and able to run a 360M-parameter model at Q8_0 in a reasonable amount of time.
Second RVV implementation	Implementation axis	A *second, different* RVV 1.0 implementation. Objectives O2 and O5 need it. Simulation can stand in for it, but the result is weaker.
Simulation fallback	Contingency	Sail or Spike reference execution, used to put bounds on behaviour if we cannot get a second physical board.

5.2 Software stack

Table 5.2 lists the tools we intend to use and why. We tried to pick each one for a property the project needs, rather than because we happen to know it.

Table 5.2: The tools we plan to use, and the reason for each.

Layer	Technology	Why this choice
Orchestration	Python 3.11+	Contract schema validation, the comparator, canary selection, reporting. Quick to iterate on analysis code.
Runner	C++17, `llama.cpp` linked as a library at a pinned commit [11]	Exact control over tokens, structured capture of numbers, and site identifiers that survive a rebuild. None of that is possible by parsing the CLI's text output.
Binary analysis	`objdump`, `readelf`, `pyelftools` (ELF/DWARF)	Maps an emitted instruction back to a source line and symbol.
Numerical replay	C++ reference reducers; NumPy for envelope assembly	Deterministic generation of legal reduction strategies.
Statistics	SciPy, `statsmodels`	Wilson intervals, bootstrap, McNemar, Kendall's τ.
Artefact store	Content-addressed files, SHA-256, Parquet for tabular traces	Nothing can be altered, and every figure can be rebuilt offline.
Reporting	Static HTML and LaTeX	No server needed. The evidence bundle will still be readable years from now.
Model	SmolLM2-360M-Instruct, GGUF, Q8_0, Apache-2.0 [12]	Small enough to iterate on a dev board, permissively licensed, and pinned by file SHA-256.

5.3 Pinned configuration for the first release

Everything in the listing below goes into the stack manifest and gets hashed, so a qualification run can be reproduced from its manifest alone.

```
runtime      llama.cpp, pinned by commit hash, linked as a
   static library
runner       vfg-llama-runner (C++17)
model        SmolLM2-360M-Instruct, GGUF, Q8_0, pinned by file
   SHA-256
decode       batch 1, 1 thread, greedy (temperature = 0), fixed
    seed
context      fixed length, fixed token budget
prompts      pre-tokenised token IDs (tokenizer cannot drift
   between runs)
compilers    GCC and Clang, both disassembled and inventoried
```

```
policies      native | forced ordered | forced unordered |
    widened ordered
targets       primary RVV 1.0 target, then a second
    implementation
```

Listing 5.1: Pinned MVP 0.1 configuration.

Quantisation stays fixed within a run

Changing the quantisation changes the spread of magnitudes that flow through a reduction, and that in turn changes how much precision the reduction loses to cancellation. If we changed quantisation and the compiler at the same time, we would not be able to tell which one caused what we saw. So within any single qualification, quantisation does not change.

5.4 Scope boundary

A proposal should be judged partly on what it decides not to do. Every item in the right-hand column of Table 5.3 could take up the entire twenty-four weeks on its own.

Table 5.3: In scope and explicitly out of scope for MVP 0.1.

In scope	Deferred
Single-node, single-thread CPU inference on RVV 1.0	Distributed and tensor-parallel inference
One runtime (`llama.cpp`) behind an adapter interface	GPU backends and kernel optimisation
One model, one quantisation, greedy decoding	Support for many model formats and samplers
Command-line tool and static reports	Multi-user web dashboard and hosted service
Minimal reproducers for upstream submission	Automatic compiler patch generation
Legality classification against the published specification	Upstreaming machine-readable rules to the RISC-V Unified Database

5.5 Repository layout

```
vfred-gate/
  vfg/                      # Python package
    contracts/              # schema, validation, rule evaluation
```

```
    scan/                  # ELF/DWARF inventory, site identification
    execute/               # local and SSH executors
    compare/               # metric computation, contract engine
    canary/                # scoring, selection, freezing, holdout split
    localise/              # teacher-forced replay, layer descent
    replay/                # legal reduction-tree sampling, envelope
    mitigate/              # ddmin, cost-ordered search
    store/                 # content-addressed artifact store
    report/                # static HTML and LaTeX emitters
  runner/                  # vfg-llama-runner (C++), llama.cpp patch set
  plans/                   # qualification plans (YAML)
  targets/                 # target manifests
  prereg/                  # timestamped preregistration documents
  docs/adr/                # architecture decision records
  tests/                   # unit, integration, injected-fault
```

Listing 5.2: Intended repository structure.

CHAPTER 6

Experimental Design and Evaluation Plan

6.1 Three experimental axes

The experiment separates three sources of variation, shown in Figure 6.1. The configurations with a forced reduction policy are controls. Nobody would ship a build like that. Their job is to take reduction order out of the picture, so that anything still different afterwards has to be coming from somewhere else.

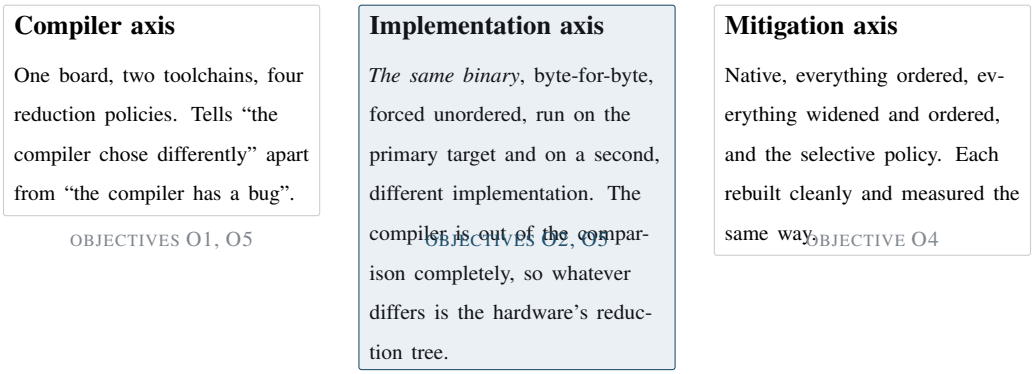


Figure 6.1: The three axes along which we vary things. The implementation axis is the strongest control we have, because the whole software stack is held byte-for-byte identical.

Table 6.1: Configurations on the compiler axis, all executed on the primary RVV target.

Toolchain	Reduction policy	Purpose
GCC	native	What production would use, checked rather than assumed
Clang	native	Production-like output, observed rather than assumed
GCC	forced ordered	Compiler control
Clang	forced ordered	Compiler control
One selected toolchain	forced unordered	Mechanism isolation
One selected toolchain	scalar left fold	Ground-truth reference

6.2 Metrics

Table 6.2: The main metrics, which ladder level each one belongs to, and which objective it serves.

Metric	Ladder level	Objective	Definition
Max absolute / relative / ULP error	Numeric (2)	O2	Deviation between baseline and candidate tensors at a layer boundary
argmax agreement rate	Decision (3)	O1, O4	Proportion of decode steps selecting the same token
Top-k set agreement	Decision (3)	O1	Jaccard overlap of the top-k logit indices
First-divergence position	Sequence (4)	O1, O3	Token index at which the two executions first differ
Token sequence exact match	Sequence (4)	O4	Proportion of cases with identical full output
Envelope containment rate	—	O2	Proportion of observed values falling inside the sampled legal envelope
Capture volume ratio	—	O3	Bytes captured by adaptive tracing versus exhaustive dumping
Toggle-confirmation rate	—	O3	Proportion of localisations confirmed by single-site intervention
Relative overhead	—	O4	Throughput of a hardening policy relative to the native candidate

6.3 Statistical procedures

Table 6.3: Statistical procedures fixed ahead of time, and what each one is for.

Procedure	Applied to
Wilson score interval [13]	Agreement proportions. Behaves better than the normal approximation when the proportion is near 0 or 1
Bootstrap interval [14]	Median first-divergence position; latency and throughput differences (paired)
McNemar's test [15]	Paired binary outcome changes between two configurations
Kendall's τ and inversion counts [16]	Preservation of rankings at ladder level 6
Effect sizes with raw counts	Reported next to every p-value

6.4 Preregistration protocol

We are preregistering the study [17] for a simple reason. In work like this there is almost always some metric somewhere that will show a difference if you go looking for one. If you choose the metric after you have seen the data, you will find one by accident and then believe it. So before any confirmatory run, the following get written down, timestamped and committed: the exact commits and toolchain versions, the case pools and the holdout split, the contract thresholds and the number of repetitions, the primary metrics and the exclusion rules, how unstable cases will be handled, which statistical procedures will be used, and the kill and rescope criteria. These can be amended, but only with a timestamp that comes before the runs the amendment affects.

6.5 Threats to validity

Table 6.4: Identified threats and the control applied to each.

Class	Threat	Control
Internal	An uncontrolled stack difference is mistaken for a reduction-order effect	Manifest diffing; fail-closed to `INCONCLUSIVE`
Internal	The first differing tensor is assumed to be the cause	Single-site toggle intervention required for every localisation
Construct	Envelope containment interpreted as proof of conformance	Sampling caveat stated at every point of use
External	Results hold only for one model at one quantisation	Reported as scope; O6 audits component transfer
External	Results hold only for one board	Implementation axis; simulation fallback where necessary
Conclusion	Metric selected after inspecting data	Preregistration with timestamped amendment discipline
Conclusion	Self-review by a team that wants one answer	External reproduction or upstream maintainer review required

Something a small team cannot fix on its own

The four of us can review each other's work, but we cannot review it independently, because we all want the same result. So we are requiring that every central claim gets checked from outside. That means either somebody outside the team reproducing it, or a maintainer of one of the upstream projects reviewing it properly. If a maintainer does not reply, we do not count that as agreement.

CHAPTER 7

Project Plan

7.1 Phase structure and gates

The schedule has six phases, each with a gate. The order was chosen carefully, and the most important thing about it is that we do not touch the language model until phase two. Phase zero is a plain reduction microbenchmark. If we cannot show the effect on a few hundred floating-point numbers, there is no reason to go looking for it inside a transformer.

Table 7.1: The six phases, what each delivers, and what it takes to pass. A phase does not start until the one before it has passed its gate.

Ph.	Weeks	Deliverable	Gate (pass condition)
0	1–2	Reduction microbenchmark; first disassembly; target manifest	Two reduction policies produce a reproducible, measurable difference on the primary target, with the responsible instruction identified in the disassembly
1	3–5	Manifests, hashing, ELF/DWARF scanner, executors	A stack is built, scanned and executed end to end from a manifest alone, and the run re-derives offline
2	6–8	Runner over pinned `llama.cpp`; free-run mode; canary format	A baseline qualification returns a verdict with a correct exit code on a frozen canary suite
3	9–14	Teacher-forced replay; layer fingerprints; causal toggle	A seeded divergence is localised to the correct instruction and confirmed by single-site intervention
4	15–19	Legal-tree sampling; envelope; ddmin; specialised builds	A selective policy is produced, rebuilt cleanly and measured against the global ordered policy
5	20–24	Holdout validation; confirmatory runs; report and paper draft	All six objectives answered, positively or negatively, with preregistered analyses

7.2 Schedule



Figure 7.1: The twenty-four week schedule. Phase 3 gets the most time because we expect localisation to be the hardest part. The red milestones are hard gates. A phase does not start until the previous gate has been passed.

7.3 Work breakdown structure

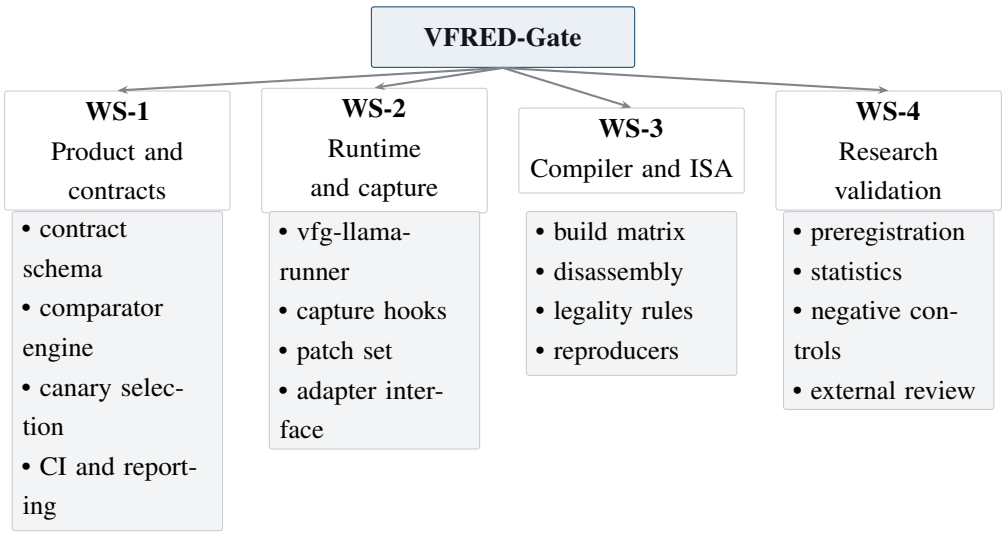


Figure 7.2: The four workstreams. WS-2 and WS-3 run in parallel from phase 2. WS-4 runs the whole way through and signs off the confirmatory analyses in phase 5.

Table 7.2: Who owns which workstream, and what each one has to deliver.

ID	Workstream	Scope	Owner
WS-1	Product and contracts	Schemas, comparator, contract engine, canary selection, CI integration, reporting	⟨assign⟩
WS-2	Runtime and capture	Runner, capture hooks, `llama.cpp` patch set, adapter interface	⟨assign⟩
WS-3	Compiler and ISA	Build matrix, disassembly, legality classification, minimal reproducers	⟨assign⟩
WS-4	Research validation	Preregistration, statistics, negative controls, external review process	⟨assign⟩

7.4 Feasibility

The obvious question about a proposal this size is whether an undergraduate team can actually deliver it. Our answer is not that we are unusually capable. It is that the system breaks into six parts, and for each of those parts, one of us has already built and shipped something very similar.

Table 7.3: Each part of the system, and the earlier work by a team member that the time estimate is based on.

Subsystem to be built	Precedent	Evidence
ISA legality classification	`riscv/riscv-unified-db`	Twelve patches merged upstream, none rejected
Reduction and collective semantics	`hpx::collectives`	Twenty-seven merged pull requests; a 960-execution measurement campaign across up to 128 localities
Canary selection and artefact commitment	CommCanary	Workload traces reduced to replayable artefacts with decision-behaviour checks
Offline re-derivation of every figure	`lfx-firstanalysis`	Published figures re-derive from committed artefacts or the build fails
Low-precision numerical measurement	`qgemm-mx`	Block-scaled FP4: bandwidth reduction versus realised speedup
Inference-engine internals	`slipstream`	Inference engine implemented from scratch, scheduling under KV-cache limits

Prior work we should mention

CommCanary is an earlier project by one of us, and it is close enough to this one that we would rather say so here than have somebody find it later. It also does canary selection and hashed artefacts. The difference is what it is looking for. CommCanary is about communication faults in distributed GPU jobs. VFRED-Gate is about numerical behaviour in a compiler and an instruction set. Some of the engineering carries over. The problem does not.

7.5 Risk register and kill criteria

Table 7.4: The risks we have identified, what happens if each one materialises, what we will do about it, and which gate the decision is made at. Each response is tied to a gate so that it does not get decided at the end under pressure.

Risk	Impact	Response	Gate
No second RVV implementation available	O2 and O5 much weaker	Fall back to Sail or Spike simulation, and say clearly that the result is weaker	2
Phase 0 finds no measurable divergence on the primary target	The premise does not hold on the available hardware	Write up a bounded negative result for that implementation, including the inputs we tried and the bound we could put on the difference. Do not go on to a twenty-week search	0
Capture hooks do not survive `llama.cpp` rebuilds	Localisation becomes unreliable	Pin the commit and keep a local patch set instead of tracking upstream	2
Localisation cannot reach instruction granularity	O3 only partly answerable	Restate O3 at the operator level and say so in the report	3
Selective hardening is not cheaper than global	The practical justification is wrong	O4 comes back negative. Still worth reporting	4
Instrumentation overhead distorts performance measurement	O4 conclusions invalid	All performance figures taken from clean specialised rebuilds only	4

Kill criteria

A negative result with clear bounds is one of the things this project can deliver. It is not a failure. If we cannot get through the phase zero gate, we will write up what we found, including the inputs we tried and the bound we managed to put on the difference, and then rescope. We are writing this down now so that the decision does not have to be made later, under deadline pressure, when it would be tempting to push on anyway.

CHAPTER 8

Expected Outcomes and Conclusion

8.1 Expected deliverables

Table 8.1: What we expect to hand over at the end, by category.

Category	Deliverable
Software	A self-hosted command-line release gate; behaviour-contract schema and engine; a dedicated C++ runner over pinned `llama.cpp`; ELF/DWARF scanner; a content-addressed artefact store with offline re-derivation
Research	A preregistered study answering the six objectives; a sampled portability envelope validated against hardware; a canary economics result with holdout validation; a measured comparison of selective against global hardening
Documentation	Project report; architecture decision records for every reversal of a recorded decision; minimal reproducers for any defect identified; an auditable evidence bundle
Community	If we find a real defect, a minimised and redacted reproducer that can be sent to the relevant upstream maintainer. We are not setting a target number, because bugs cannot be scheduled

8.2 Definition of done

Definition of done

Somebody who has never seen the repository can clone it, build the two stacks, scan them, run the frozen canary suite, get a contract verdict, follow a failure from the token that went wrong down to a machine instruction, get a mitigation with a measured cost, and rebuild every figure in the report offline from the checksummed artefacts.

8.3 Broader applicability

We have scoped this to RISC-V, but the underlying mechanism is not specific to RISC-V. Any architecture with a vectorised floating-point reduction will show the same effect. We chose RISC-V because its specification says explicitly what the hardware is allowed to do, and that is what makes the legal-versus-illegal question answerable at all. The contract format and the localisation procedure should, in principle, carry over to any situation where a reduction order is left unspecified, which includes tensor-parallel all-reduce and federated aggregation. Objective O6 is there to measure how much of that transfer actually holds, rather than assuming it does.

8.4 Conclusion

Version 1.0 of the RISC-V vector extension lets an unordered floating-point reduction pick its own evaluation tree and its own intermediate precision. Because floating-point addition is not associative, two implementations that both follow the specification can still disagree on the numbers. That disagreement travels through the reductions that make up transformer inference and can change which token the model emits. Right now, certification checks the processor, benchmarks check the model, and nobody is checking the layer in between.

What we have proposed is a release gate for that layer: a self-hosted tool that takes a written description of the behaviour that has to be preserved, returns a verdict that a CI system can use, traces the first real incompatibility to a machine instruction and confirms it by intervention, and looks for the cheapest repair instead of forcing everything to be deterministic.

None of this has been built yet. What we have done is understand the problem well enough to scope it, decide what to leave out, write six objectives that can each come back negative, and attach a schedule with gates and kill criteria already written down. The claims we are not making are listed in Section 3.4. We intend to stick to that list.

Bibliography

[1] RISC-V International, "RISC-V "V" Vector Extension, Version 1.0." `https://github.com/riscv/riscv-v-spec`, 2021. Ordered and unordered single-width floating-point sum reductions (`vfredosum.vs`, `vfredusum.vs`).

[2] vLLM Project, "Batch Invariance and Reproducibility in vLLM." `https://docs.vllm.ai`, 2025.

[3] Anonymous, "Deterministic Inference across Tensor Parallel Sizes That Eliminates Numerical Inconsistency," *arXiv preprint arXiv:2511.17826*, 2025.

[4] R. Gond *et al.*, "LLM-42: Enabling Determinism in LLM Inference with Verified Speculation," *arXiv preprint arXiv:2601.17768*, 2026.

[5] NVIDIA Corporation, "Polygraphy: A Deep Learning Inference Prototyping and Debugging Toolkit." `https://github.com/NVIDIA/TensorRT/tree/main/tools/Polygraphy`, 2025.

[6] T. Zhu, "From Expert Reduction to Behavioral Divergence: Tracing Numerical State through Sparse MoE Inference," *arXiv preprint arXiv:2607.28097*, 2026.

[7] RISC-V International, "RISC-V Unified Database." `https://github.com/riscv-software-src/riscv-unified-db`, 2025.

[8] RISC-V International, "RISC-V Architectural Certification Tests." `https://github.com/riscv-non-isa/riscv-arch-test`, 2025.

[9] LLVM Project, "Auto-Vectorization in LLVM." `https://llvm.org/docs/Vectorizers.html`, 2026. Floating-point reduction and reassociation requirements.

[10] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[11] ggml-org, "llama.cpp: LLM inference in C/C++." `https://github.com/ggml-org/llama.cpp`, 2026.

[12] Hugging Face, "SmolLM2-360M-Instruct-GGUF." `https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct-GGUF`, 2025. Apache-2.0 licensed.

[13] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[14] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[15] Q. McNemar, “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[16] M. G. Kendall, “A New Measure of Rank Correlation,” *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.

[17] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, “The Preregistration Revolution,” in *Proceedings of the National Academy of Sciences*, vol. 115, pp. 2600–2606, 2018.

[18] IEEE, “IEEE Standard for Floating-Point Arithmetic,” *IEEE Std 754-2019*, 2019.

[19] D. Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[20] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd ed., 2002.

CHAPTER A

Illustrative Behaviour Contract

Below is an annotated example of what we intend the contract format to look like. It is a design. The schema has not been implemented.

```
contract_version: 0.1
model:
  name: SmolLM2-360M-Instruct
  quantisation: Q8_0
  sha256: e71a44...              # model is pinned; never
     inferred

compatibility_level: decision    # ladder level 3 (see Figure
   4.1)

rules:
  - id: argmax_agreement
    metric: argmax_agreement_rate
    threshold: ">= 0.9990"
    scope: all_decode_steps

  - id: topk_overlap
    metric: topk_jaccard
    k: 5
    threshold: ">= 0.9950"

  - id: token_sequence_exact
    metric: sequence_exact_match_rate
    threshold: ">= 0.9900"

evidence_requirements:
  repetitions: 5
  stability: required            # unstable cases quarantined,
     not averaged
  uncontrolled_differences: none # any non-none result forces
     INCONCLUSIVE

on_insufficient_evidence: INCONCLUSIVE   # fail closed; never
   silently PASS
```

Listing A.1: Example behaviour contract (illustrative).

CHAPTER B

Illustrative Verdict Output

This is a mock-up

The transcript below shows the output format we are aiming for. It did not come from a real run. Nothing in the system described in this report has been implemented.

```
$ vfg qualify --plan plans/gcc-vs-clang.yaml --output runs/

  baseline    gcc-15.2    -O3 -march=rv64gcv      sha 4f1c9a...
  candidate   clang-22    -O3 -march=rv64gcv      sha 8b02de...
  model       SmolLM2-360M-Instruct  Q8_0         sha e71a44...
  target      rvv-target RVV 1.0  VLEN=256

  uncontrolled stack differences ............... none
  cases 240/240    repetitions 5    stability ok

  RULE  logit_exact_equality ......... 0.9958    PASS
  RULE  argmax_agreement ............. 0.9791    FAIL   (>=
     0.9990)
  RULE  token_sequence_exact ........ 0.9625    FAIL   (>=
     0.9900)

  VERDICT  FAIL                                          exit 2

$ vfg localize --qualification runs/<id> --failure first

  first divergent token ....... position 31, case c0117
  teacher-forced replay ....... 31 tokens, identical inputs to
     both
  first divergent layer ....... blk.14.ffn_norm
  operator .................... rms_norm_f32
  static site ................. ggml-cpu/ops.cpp:2184
  instruction ................. vfredusum.vs v8,v12,v0  @ 0
     x00014e2c
  causal confirmation ......... site forced ordered ->
     difference gone

  CAUSE  conformant, contract-breaking          confidence
     high
```

Listing B.1: Illustrative output of a qualification and localisation run. Mock-up.

Chapter C

Verified Demonstration Programs

The programs in this appendix are different from the mock-up in Appendix B. We compiled and ran them on an ordinary x86-64 machine with GCC at `-O0`, and the output shown is what they actually printed. They demonstrate the mechanism the whole proposal rests on, and they do it without any RISC-V hardware, because the effect is a property of IEEE 754 arithmetic [18, 19, 20] and shows up on any machine that follows the standard.

C.1 Non-associativity of floating-point addition

```c
#include <stdio.h>
int main(void) {
    double a = 1e20, b = -1e20, c = 3.14;
    printf("(a + b) + c = %.10f\n", (a + b) + c);
    printf("a + (b + c) = %.10f\n", a + (b + c));
    return 0;
}
```

Listing C.1: Demonstration of non-associativity.

```
(a + b) + c = 3.1400000000
a + (b + c) = 0.0000000000
```

Listing C.2: Verified output of Listing C.1.

Both answers are correctly rounded. In the second grouping, 3.14 is smaller than half the ULP at 10^{20}, so it is absorbed before the cancellation can bring it back.

C.2 Sequential versus tree reduction

```c
float seq_sum(const float *x, int n) {
    float s = 0.0f;
    for (int i = 0; i < n; i++) s += x[i];
    return s;
}
```

```c
float tree_sum(const float *x, int n) {
    if (n == 1) return x[0];
    int h = n / 2;
    return tree_sum(x, h) + tree_sum(x + h, n - h);
}
/* n = 2^20 values drawn uniformly from [0,1), srand(42) */
```

Listing C.3: Left fold compared with a pairwise reduction tree.

```
sequential (left fold) : 524421.375000
pairwise   (tree)      : 524416.000000
difference             : 5.375000
```

Listing C.4: Verified output of Listing C.3.

The sequential version loses more because once the running total is large, every further addition of a small value loses some precision. The tree keeps adding values of similar size to each other, so it loses less.

C.3 Absorption and ULP growth

```
1e20          1e+20     sign=0 exp=193 mantissa=0x2D78EC
3.14          3.14      sign=0 exp=128 mantissa=0x48F5C3
1e20+3.14     1e+20     sign=0 exp=193 mantissa=0x2D78EC

1e20 + 3.14 == 1e20 ? YES
gap between adjacent floats near 524288 is 0.0625
```

Listing C.5: Verified bit-level inspection of `binary32` absorption.

The significand of `1e20+3.14` is bit-for-bit identical to that of `1e20`. The addition did not change anything at all.

C.4 Reduction order changing an argmax

This program builds two candidate logits from the same set of products, arranged in two different orders, with the second one ahead by 1×10^{-5}. Adding them up sequentially and adding them up as a tree gives different winners.

```c
srand(7);
for (int i = 0; i < n; i++) p0[i] = ((float)rand()/RAND_MAX -
   0.5f) * 20.0f;
```

```
for (int i = 0; i < n; i++) p1[i] = p0[n - 1 - i];    /* same
    values, reordered */
p1[0] += 0.00001f;                                     /* token 1
     truly ahead */

printf("left fold : logit0=%.6f  logit1=%.6f  -> emits token %d
    \n",
        seq_sum(p0,n),  seq_sum(p1,n),  seq_sum(p1,n)  > seq_sum
           (p0,n));
printf("tree      : logit0=%.6f  logit1=%.6f  -> emits token %d
    \n",
        tree_sum(p0,n), tree_sum(p1,n), tree_sum(p1,n) >
           tree_sum(p0,n));
```

Listing C.6: Summation order altering the emitted token.

```
left fold : logit0=140.687103  logit1=140.686768  -> emits
    token 0
tree      : logit0=140.686935  logit1=140.686951  -> emits
    token 1
```

Listing C.7: Verified output of Listing C.6.

Why this appendix is here

The model emits a different token. The only thing that changed was the order in which one sum was accumulated, and both orders are legal. That is the whole premise of the project, in about forty lines of C, and it can be reproduced on any machine before any RISC-V hardware is involved.

CHAPTER D

Glossary

Term	Definition
Absorption	A small value added to a much larger one being discarded entirely by rounding.
Argmax	The index of the largest element of a vector; under greedy decoding, the emitted token.
Autoregressive	Generation in which each emitted token is appended to the input for the next step.
Behaviour contract	A versioned, schema-validated declaration of the compatibility level and thresholds that a release must satisfy.
Canary	A test case selected for its propensity to expose a contract failure, so that a small suite predicts a large one.
Content-addressed	Stored under the hash of its own contents, so that any modification is detectable.
ddmin	Delta debugging minimisation: reduction of a failing input to a minimal still-failing subset [10].
Fail closed	Defaulting to blocking a release when evidence is absent or inadequate.
Greedy decoding	Emitting the argmax at every step, with no sampling; used here to remove stochasticity from experiments.
Latent divergence	Internal state that already differs between two executions while the emitted tokens remain identical.
Preregistration	Committing the experimental design, thresholds and analysis plan before running the confirmatory experiment [17].
Reduction	Collapse of an array to a scalar under an operator associative over the reals.
Teacher forcing	Supplying a known token history to a model in place of its own generated output.

Term	Definition
ULP	Unit in the last place: the gap between adjacent representable floating-point values.